

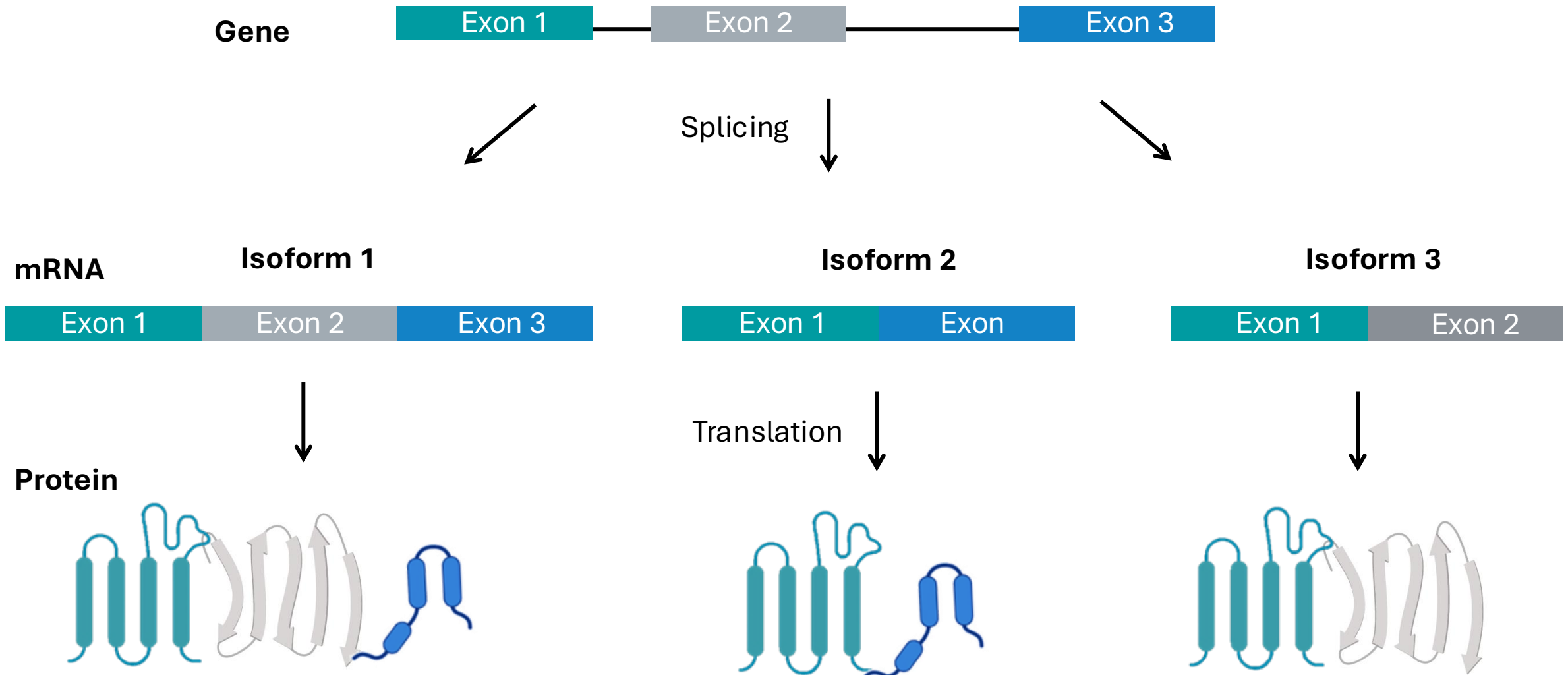
Differential transcript usage (DTU) analysis

Same gene, different message?

Code Club
Maria Skjøtt Hansen
February 9, 2026

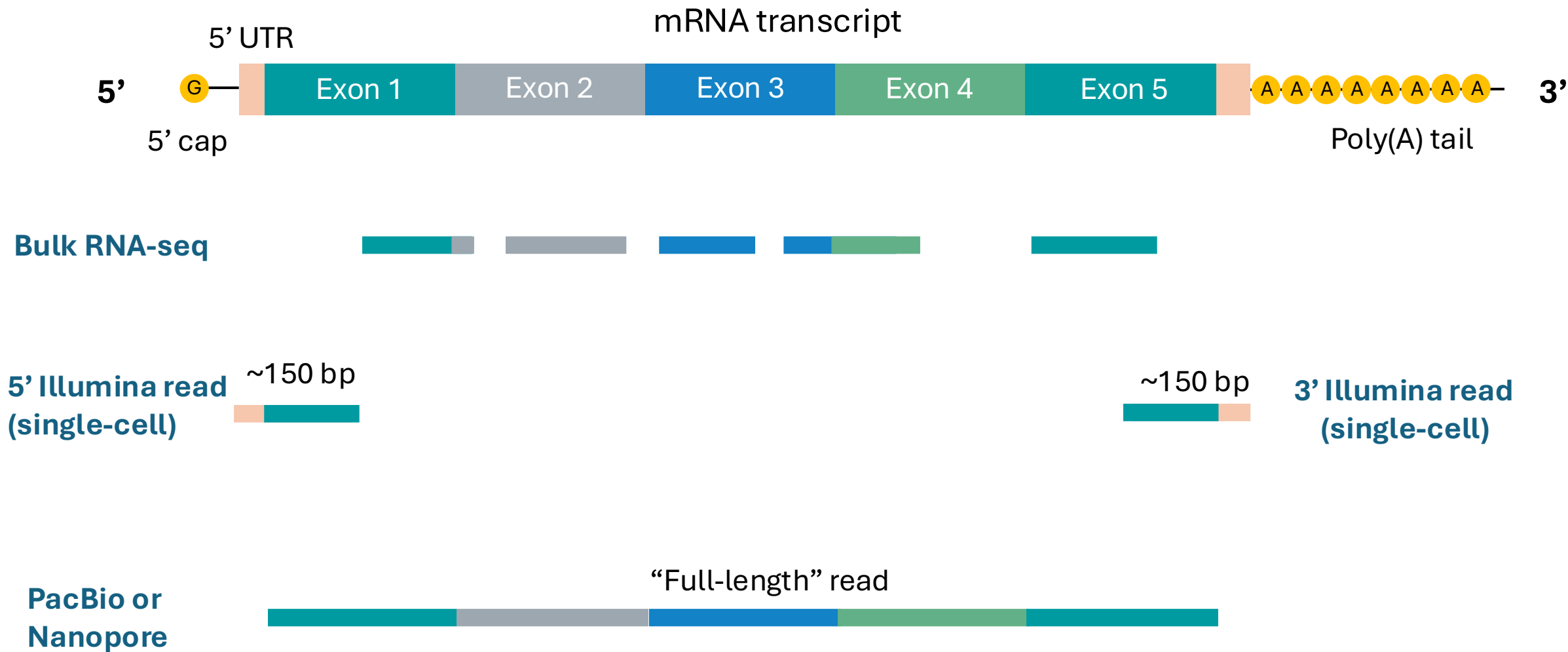


Alternative splicing results in different protein isoforms

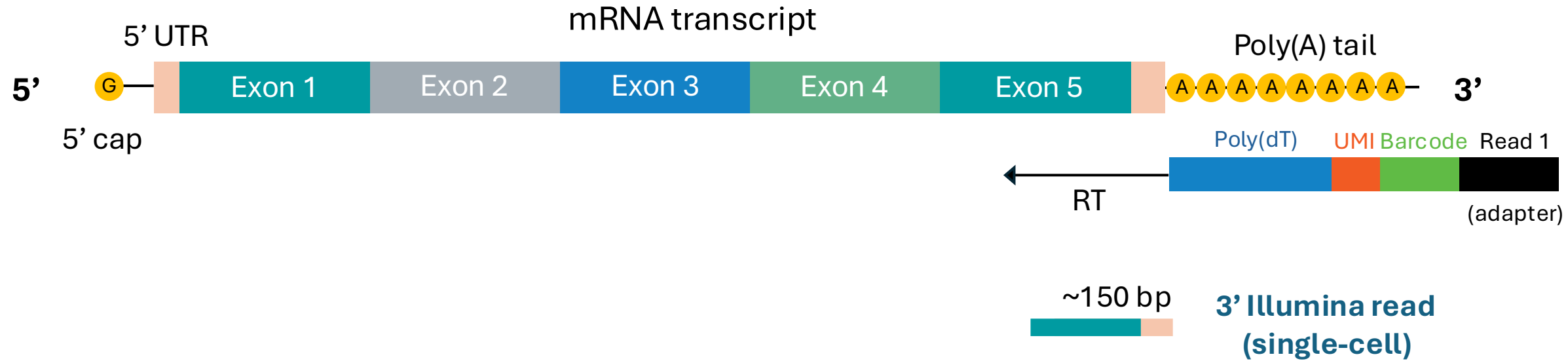


Protein isoforms can have different binding capacities → different functional properties

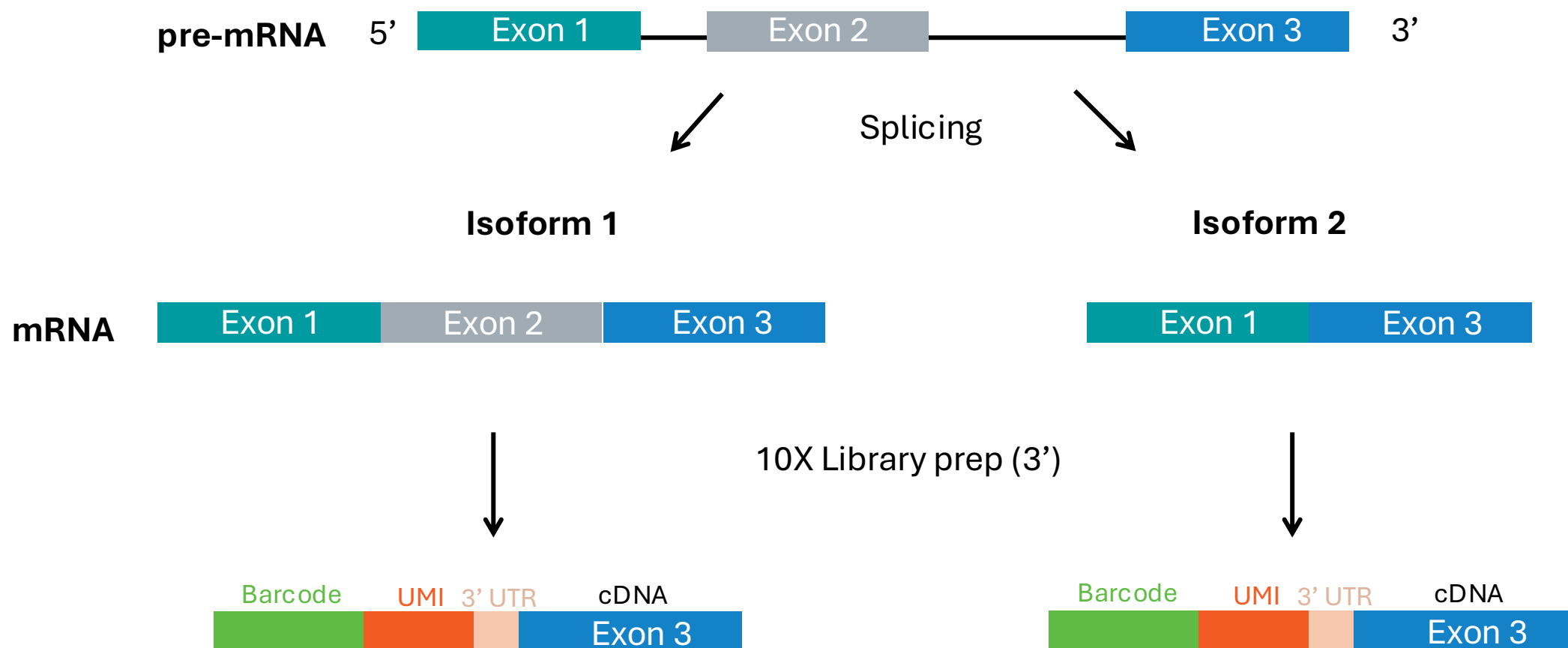
No isoform resolution in regular RNA sequencing



Only the end of the transcript is captured in single-cell RNA-seq

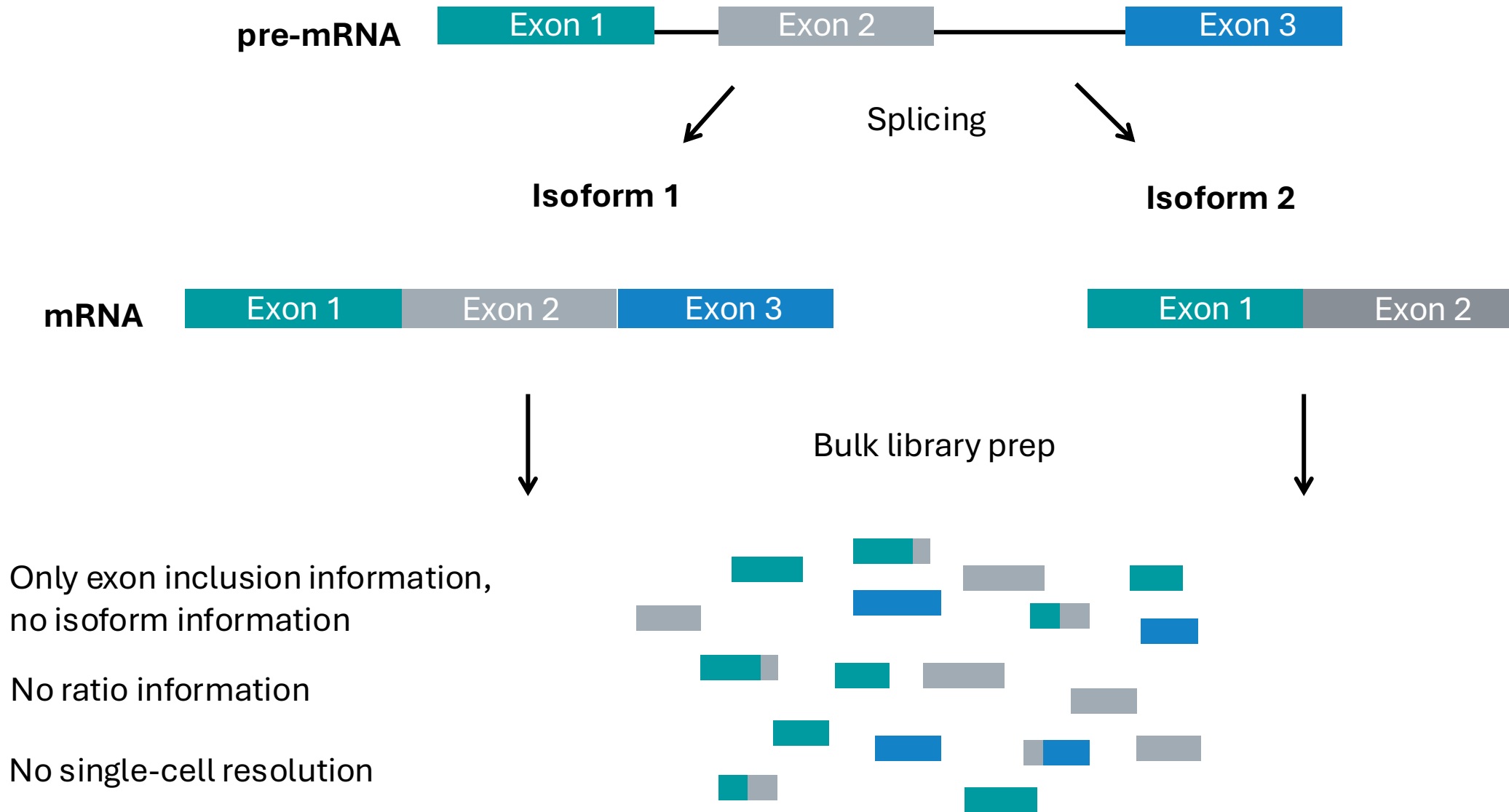


No isoform resolution in single-cell short-read

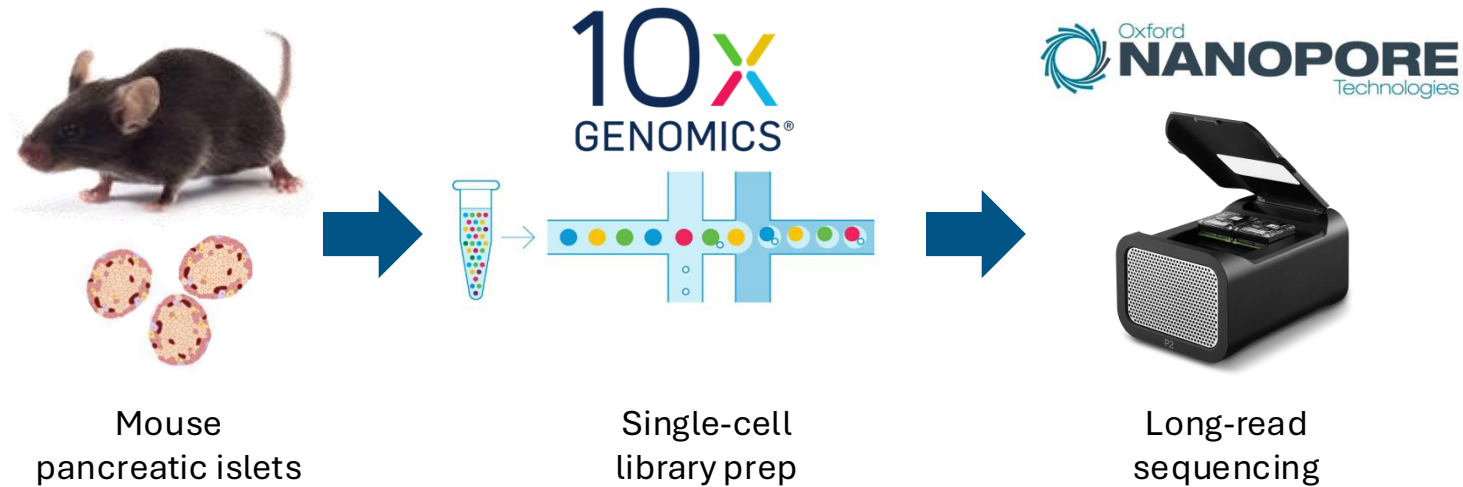


Reads will only cover first exon, or first exon-exon junction at most

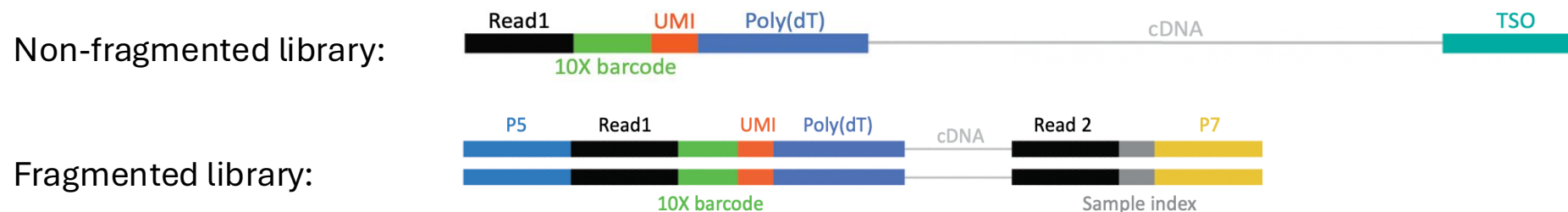
No isoform resolution in bulk short-read

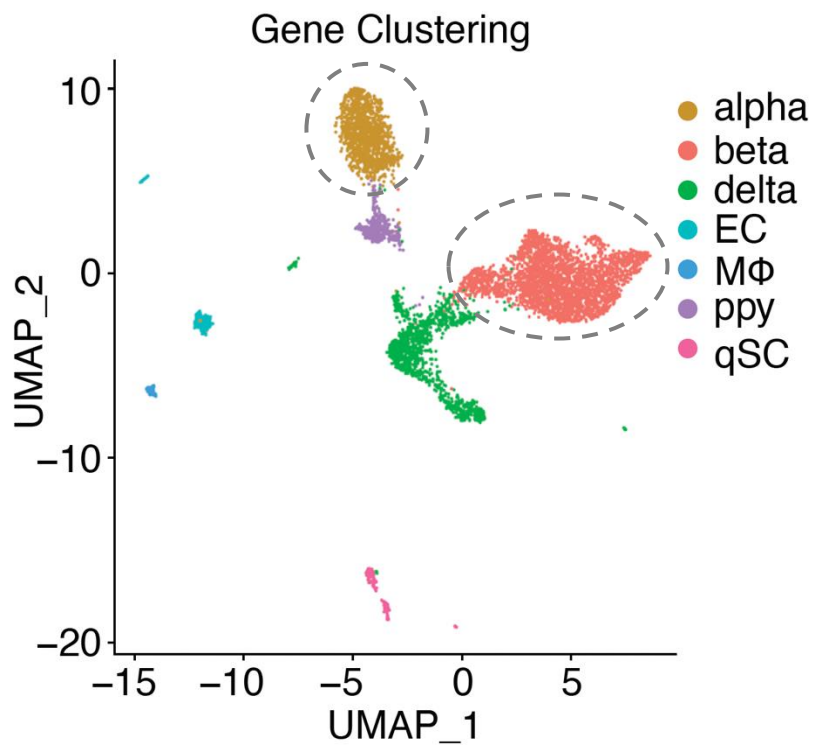


Single-cell long-read sequencing pancreatic islets



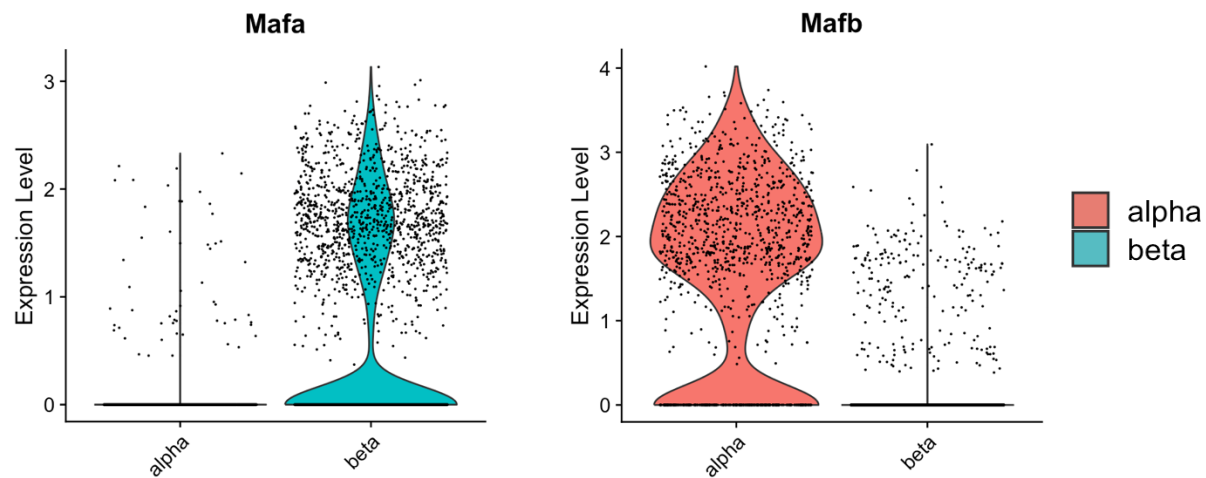
Don't fragment the library you want to long-read sequence!



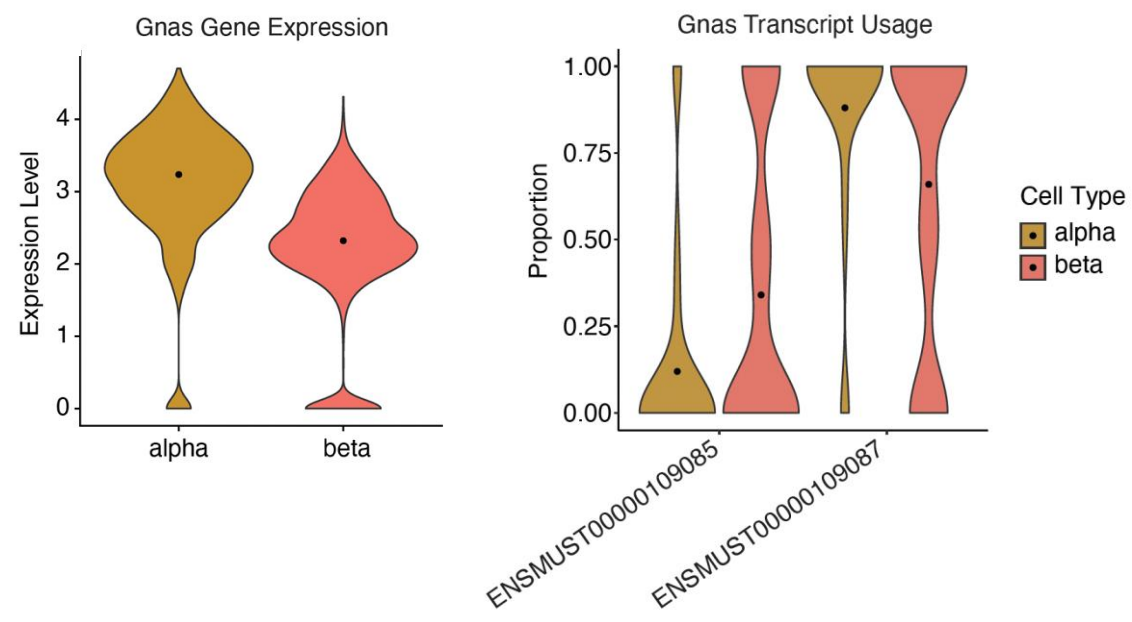


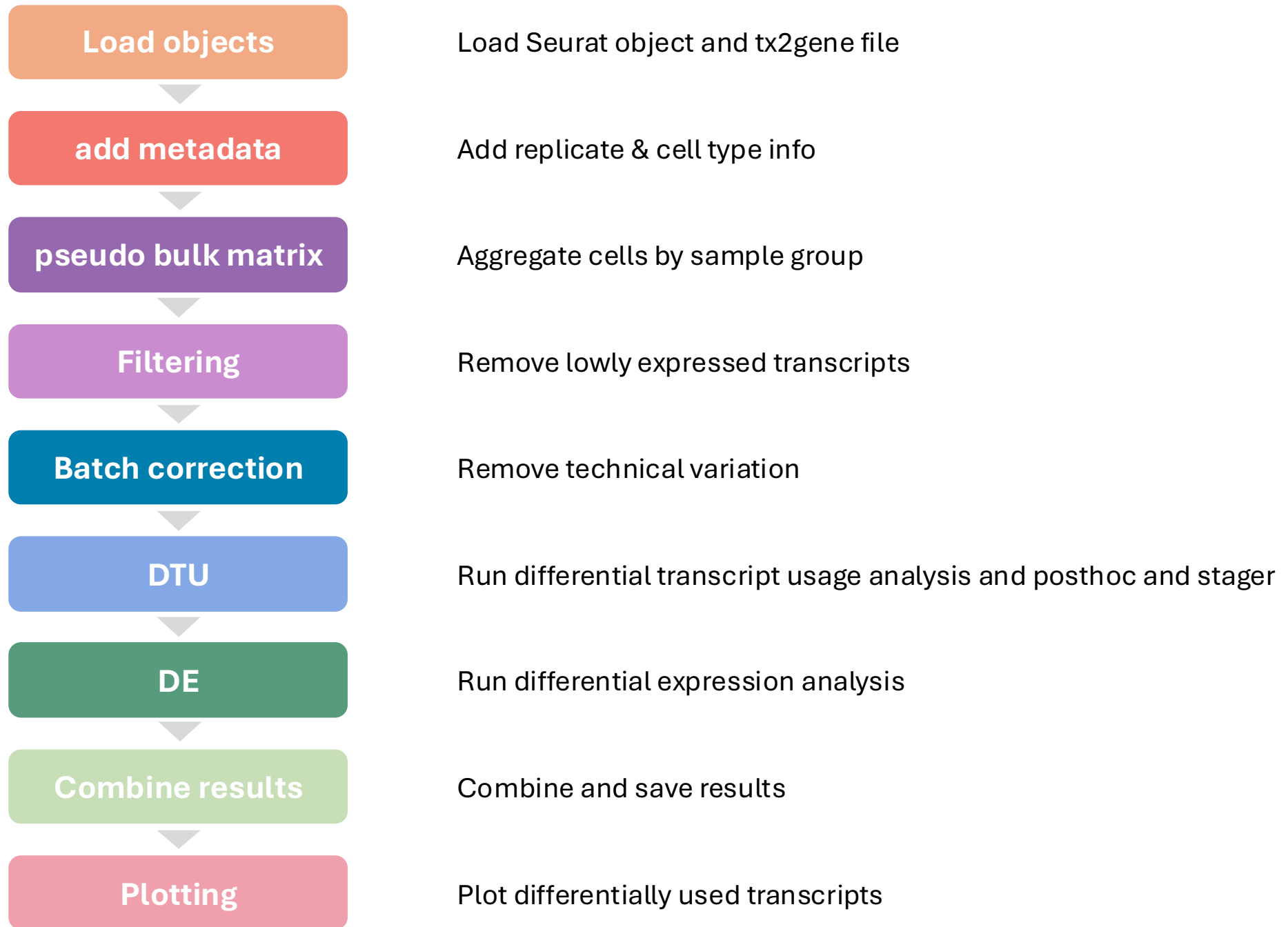
DE/DGE analysis

Do alpha and beta cells express the same genes?



Do alpha and beta cells express the same isoforms?





Load objects

```
so <- readRDS("/Users/hansmari/Documents/long_read_sequencing/long_read_methods_paper/objects/5prime_tx_merged.rds")
tx2gene <- readRDS("/Users/hansmari/Documents/long_read_sequencing/long_read_methods_paper/reference/tx2gene_longread.rds")
```

add metadata

```
# Look at original IDs
```

```
unique(so$orig.ident)
```

```
> unique(so$orig.ident)
```

```
[1] "MSH_1_mod_5prime_pass" "islet_5p_mod"
```

```
# Extract metadata
```

```
metadata <- so@meta.data
```

```
> head(so@meta.data[,c(1,2,4,29,30,39)])
```

```
# Map original names → Rep labels
```

```
replicate_map <- c(
  "MSH_1_mod_5prime_pass" = "Rep1",
  "islet_5p_mod"          = "Rep2"
)
```

	orig.ident	nCount_RNA	nCount_TRANSCRIPTS	RNA_barcode_celltype	cluster_celltype	replicate
AAACCTGAGCGCCTCA_1	MSH_1_mod_5prime_pass	2112	1781	beta	0_beta	Rep1
AAACCTGCATGGTCTA_1	MSH_1_mod_5prime_pass	1032	940	macrophage	13_macrophage	Rep1
AAACCTGGTAGCTCCG_1	MSH_1_mod_5prime_pass	1880	1524	beta	3_beta	Rep1
AAACCTGGTGGGTATG_1	MSH_1_mod_5prime_pass	1811	1557	beta	7_beta	Rep1
AAACCTGTCAACCAAC_1	MSH_1_mod_5prime_pass	1373	1078	beta	3_beta	Rep1
AAACCTGTCCCGACTT_1	MSH_1_mod_5prime_pass	1983	1698	beta	2_beta	Rep1

```
# Create replicate column
```

```
metadata$replicate <- replicate_map[metadata$orig.ident]
```

```
so <- AddMetaData(so, metadata = metadata)
```

Why pseudo bulk?

Why single-cell counts are tricky for DTU:

High dropout in single-cell long-read data (sparse data):

Many transcripts are **zero in most cells**, even if they are expressed. Sparse matrices make statistical models unstable and unreliable.

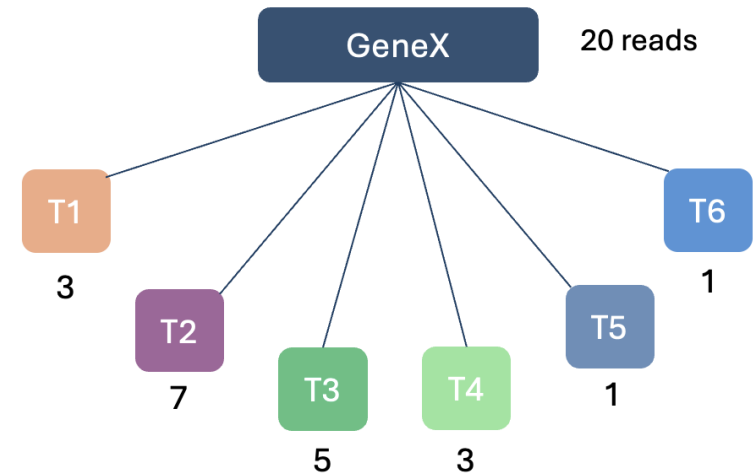
High variability/noise in single-cell data:

Single cells have biological variability **and technical noise**.

DTU models (Dirichlet-multinomial) assume counts are sufficiently large per sample to estimate transcript proportions.

Pseudo replication

When you treat non-independent measurements as independent (e.g. cells in a sample), you **inflate statistical power** which gives misleading results.



A **pseudo bulk** approach treats the biological sample, rather than individual cells, as the unit of replication, reducing pseudo replication and yielding more robust and reproducible results.

Pseudo bulk matrix

Create sample groups

```
# Combine replicate + cell type
so$sample_group <- paste0(
  so$replicate, "--", so$new_identity
)
```

Make pseudo bulk matrix:

```
pseudobulk <- PseudobulkExpression(
  so,
  layer = "counts",
  method = "aggregate",
  assay = "TRANSCRIPTS",
  group.by = "sample_group"
)[["TRANSCRIPTS"]]
```

Make metadata data frame:

```
pd <- data.frame(
  id = colnames(pseudobulk),
  group = sub(".*--", "", colnames(pseudobulk))
)
```

```
> unique(so$sample_group)
[1] "Rep1--beta" "Rep1--alpha" "Rep2--beta" "Rep2--alpha"
```

```
> head(pseudobulk)
6 x 4 sparse Matrix of class "dgCMatrix"
      Rep1--alpha Rep1--beta Rep2--alpha Rep2--beta
ENSMUST000000000001      83      496      103      303
ENSMUST000000000010      .      .      .      .
ENSMUST000000000028      .      5      3      4
ENSMUST000000000049      2      .      1      1
ENSMUST000000000058     10      3      9      4
ENSMUST000000000080     63     156     154     144
```

```
> pd
      id group
1 Rep1--alpha alpha
2 Rep1--beta  beta
3 Rep2--alpha alpha
4 Rep2--beta  beta
```

Filtering

Keep transcripts that are expressed in **5% of cells** in **at least one sample**.

1) Pull out counts from seurat object:

```
counts <- GetAssayData(  
  so,  
  assay = "TRANSCRIPTS",  
  layer = "counts"  
)
```

```
> head(counts, n=5)  
5 x 3948 sparse Matrix of class "dgCMatrx"  
  [[ suppressing 77 column names 'AAACCTGAGCGCCTCA_1', 'AAACCTGGTAGCTCCG_1', 'AAACCTGGTGGGTATG_1' ... ]]  
  
ENSMUST00000000001 1 . . . . . 1 . . . 1 1 . . . . . 1 . . 1 . . .  
ENSMUST00000000010 . . . . . . . . . . . . . . . . . . . . . .  
ENSMUST00000000028 . . . . . . . . . . . . . . . . . . . . . .  
ENSMUST00000000049 . . . . . . . . . . . . . . . . . . . . . .  
ENSMUST00000000058 . . . 1 . . . . . . . . . . . . . . . . . 1 . . .  
  
ENSMUST00000000001 . . . . .  
ENSMUST00000000010 . . . . .  
ENSMUST00000000028 . . . . .  
ENSMUST00000000049 . . . . .  
ENSMUST00000000058 . . . . .
```

2) Pull out sample groups:

```
samples_use <- unique(so$sample_group)
```

```
> samples_use  
[1] "Rep1--beta" "Rep1--alpha" "Rep2--beta" "Rep2--alpha"
```

Filtering

3) For each sample, find % of cells expressing each transcript

```
samples_use <- unique(so$sample_group)

pct_list <- lapply(samples_use, function(sample) {

  # Find cells belonging to this sample
  cell_idx <- which(so$sample_group == sample)

  # Subset counts to those cells
  mat_sub <- counts[, cell_idx, drop = FALSE]

  # Convert to expressed / not expressed and compute %
  pct_cells <- Matrix::rowMeans(mat_sub > 0) * 100

  # Store as data frame
  df <- data.frame(pct_cells)
  colnames(df) <- sample

  return(df)
})
```

4) Combine into one table:

```
pct_expression <- do.call(cbind, pct_list)
```

How many transcripts are we working with?

```
length(rownames(pct_expression)) #69994
```

```
> str(pct_list)
List of 4
 $ :'data.frame':      69994 obs. of  1 variable:
  ..$ Rep1--beta: num [1:69994] 18.918 0 0.216 0 0.13 ...
 $ :'data.frame':      69994 obs. of  1 variable:
  ..$ Rep1--alpha: num [1:69994] 8.246 0 0 0.209 1.044 ...
 $ :'data.frame':      69994 obs. of  1 variable:
  ..$ Rep2--beta: num [1:69994] 47.856 0 0.903 0.226 0.903 ...
 $ :'data.frame':      69994 obs. of  1 variable:
  ..$ Rep2--alpha: num [1:69994] 34.177 0 1.266 0.422 3.376 ...
```

```
> pct_expression
```

	Rep1--beta	Rep1--alpha	Rep2--beta	Rep2--alpha
ENSMUST000000000001	18.91774892	8.2463466	47.8555305	34.1772152
ENSMUST000000000010	0.00000000	0.0000000	0.0000000	0.0000000
ENSMUST000000000028	0.21645022	0.0000000	0.9029345	1.2658228
ENSMUST000000000049	0.00000000	0.2087683	0.2257336	0.4219409
ENSMUST000000000058	0.12987013	1.0438413	0.9029345	3.3755274
ENSMUST000000000080	6.40692641	6.1586639	24.3792325	42.1940928
ENSMUST000000000087	1.08225108	0.7306889	1.5801354	3.3755274

Filtering

5) Filter by minimum percent of cells (5% of cells in at least one replicate)

```
percent_cells <- 5  
  
keep_pct <- apply(pct_expression, 1, max) >= percent_cells  
  
pct_expression_filtered <- pct_expression[keep_pct, ]
```

How many transcripts do we have left after filtering?

```
length(rownames(pct_expression_filtered)) #10926
```

6) Filter pseudobulk counts:

```
keep_transcripts <- rownames(pct_expression_filtered)  
pseudobulk_filtered <- pseudobulk[keep_transcripts, ]
```

Batch correction

Batch correction overview:

- 1) Remove transcripts with zero counts across an entire batch.
- 2) Remove zero-variance transcripts (before batch correction).
- 3) PCA plot (uncorrected).
- 4) Batch correction.
- 5) Remove zero-variance transcripts again (after batch correction).
- 6) PCA plot (corrected).

The PCA plotting will throw an error if there are transcripts with zero variance in there! That's why you have to remove those twice (before making the PCA plots).

Batch correction

1) Remove transcripts with zero counts across an entire batch:

We've already filtered out transcripts that are not expressed in any of the samples. (5% of cells in at least one replicate required). However, the batch correction cannot handle features that are zero in an entire batch. So, we must filter out transcripts with 0 across a replicate:

```
# Get list of replicates
reps <- unique(pd$replicate)

# Check for non-zero counts in each replicate
keep <- sapply(reps, function(r) {
  rowSums(pseudobulk_filtered[, pd$replicate == r, drop = FALSE]) > 0
})

# Keep transcripts with non-zero counts in all replicates
keep_all <- rowSums(keep) == length(reps)

pseudobulk_filtered <- pseudobulk_filtered[keep_all, ]
```

2) Remove zero-variance transcripts (before batch correction).

```
# Remove transcripts with identical counts across all samples
pseudobulk_filtered <- pseudobulk_filtered[
  apply(pseudobulk_filtered, 1, var) > 0,
]
```

Batch correction

3) PCA plot (uncorrected sample)

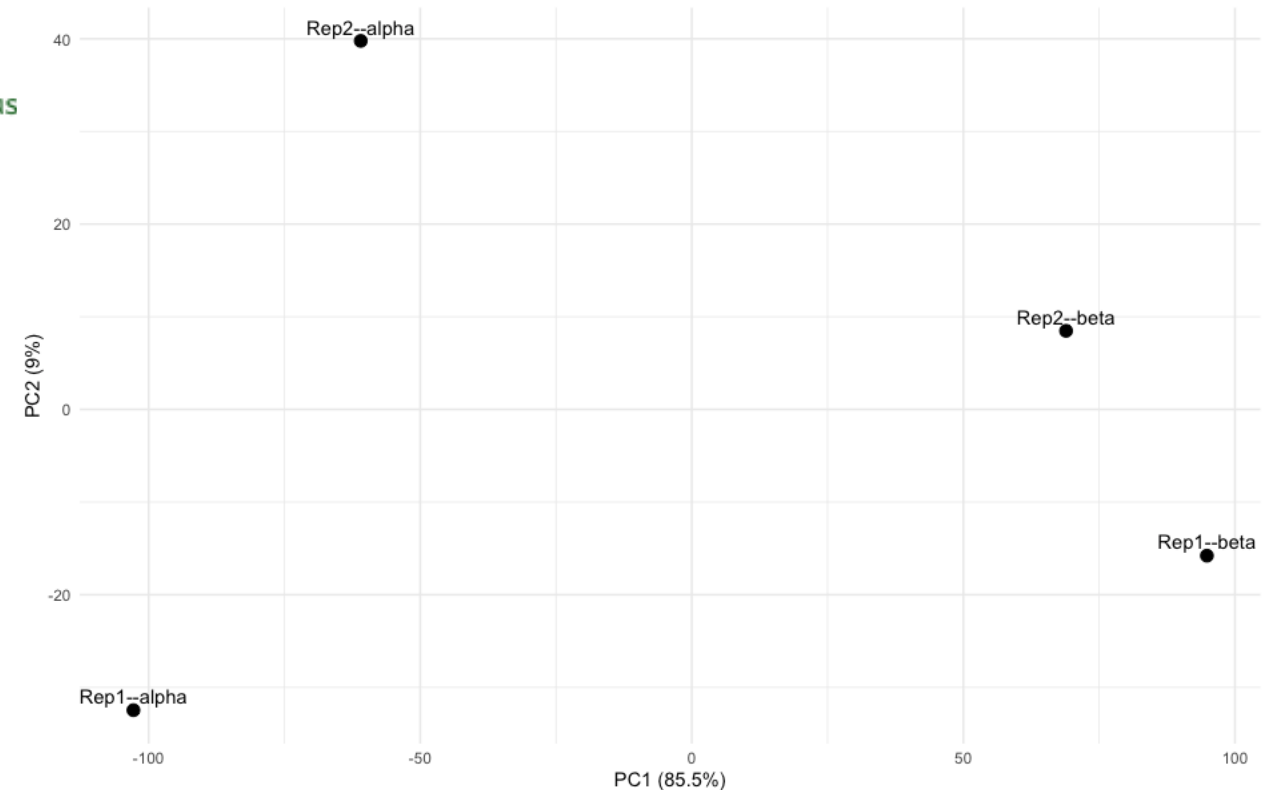
```
# log-transform for visualization
log_counts <- log1p(pseudobulk_filtered) # log(count + 1)
pca_res <- prcomp(t(log_counts), scale. = TRUE) # transpose: samples as

# Percent variance explained
var_explained <- 100 * (pca_res$sdev^2 / sum(pca_res$sdev^2))

# Prepare data frame for plotting
pca_df <- data.frame(
  PC1 = pca_res$x[, 1],
  PC2 = pca_res$x[, 2],
  sample = colnames(log_counts)
)

# Plot PCA
p <- ggplot(pca_df, aes(x = PC1, y = PC2, label = sample)) +
  geom_point(size = 3) +
  geom_text(vjust = -0.5) +
  theme_minimal() +
  xlab(paste0("PC1 (", round(var_explained[1], 1), "%)")) +
  ylab(paste0("PC2 (", round(var_explained[2], 1), "%)"))

print(p)
```



Batch correction

4) Run batch correction

```
# Run ComBat-seq batch correction:
pseudobulk_corrected <- ComBat_seq(
  counts = pseudobulk_filtered,
  batch  = pd$replicate, # replicate info
  group  = pd$group      # condition / cell type info
)
```

5) Remove zero-variance transcripts again (after batch correction).

```
pseudobulk_corrected <- pseudobulk_corrected[
  apply(pseudobulk_corrected, 1, var) > 0,
]
```

Batch correction

6) PCA plot (on batch corrected sample).

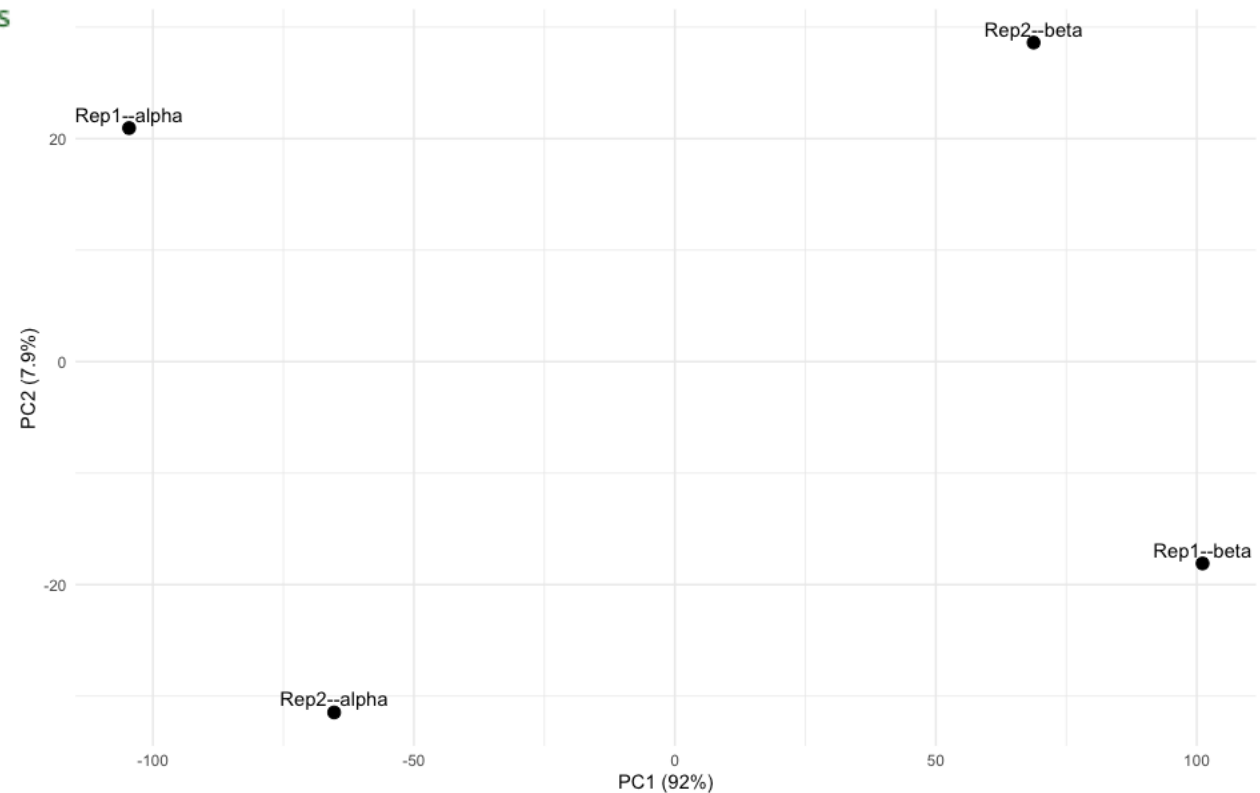
```
# PCA
log_counts <- log1p(pseudobulk_corrected) # log(count + 1)
pca_res <- prcomp(t(log_counts), scale. = TRUE) # transpose: samples

# Percent variance explained
var_explained <- 100 * (pca_res$sdev^2 / sum(pca_res$sdev^2))

# Prepare data frame for plotting
pca_df <- data.frame(
  PC1 = pca_res$x[, 1],
  PC2 = pca_res$x[, 2],
  sample = colnames(log_counts)
)

# Plot PCA
p <- ggplot(pca_df, aes(x = PC1, y = PC2, label = sample)) +
  geom_point(size = 3) +
  geom_text(vjust = -0.5) +
  theme_minimal() +
  xlab(paste0("PC1 (", round(var_explained[1], 1), "%)")) +
  ylab(paste0("PC2 (", round(var_explained[2], 1), "%)"))

print(p)
```



DTU

```
dtu_res <- DTUrtle::run_drimseq(  
  counts = pseudobulk,  
  tx2gene = tx2gene,  
  pd = pd,  
  id_col = "id",  
  cond_col = "group"  
)
```

```
> pd  
      id replicate group  
1 Rep1--alpha   Rep1 alpha  
2 Rep1--beta   Rep1  beta  
3 Rep2--alpha   Rep2 alpha  
4 Rep2--beta   Rep2  beta
```

DE (Differential Expression)

“Is this gene more or less expressed?”

DTU (Differential Transcript Usage)

“Is this gene using different isoforms/transcripts?”

“Inside this gene, do the percentages of transcripts change between groups?”

Condition	Tx1	Tx2
Alpha	80%	20%
Beta	30%	70%

DRIMSeq

A statistical package designed for DTU analysis.
(does the actual math part)

dTurtle

A wrapper/helper package for DRIMSeq
Runs Drimseq and does filtering and postprocessing



DTU

posthoc_and_stager

Identify significant genes using posthoc_and_stager function (part of the DTUrtle R package):

```
dtu_res <- posthoc_and_stager(dturtle = dtu_res, ofdr = 0.05)
```

It runs two-staged testing:

Stage 1 – gene level:

Test each gene as a whole and identify genes **that have evidence of DTU**.

Stage 2 – transcript level:

For only the genes that passed stage 1, test each transcript and find significant transcripts.

NB! It's possible to find significant genes with no significant individual transcripts!

If the gene has several transcripts, e.g. T1, T2, T3, that all shift slightly in opposite directions between conditions, the relative proportions **within** the gene change significantly (DTU). But the individual transcripts might not be significant on their own.

Posthoc_and_stager doesn't filter anything out of the dtu object, but it adds an FDR table as well as significant genes and significant transcripts:

```
dtu_res$FDR_table  
dtu_res$sig_gene  
dtu_res$sig_tx
```

DTU

Without filtering and batch correction:

Drimseq retained 30275 of 56171 features and removed 25896.
Posthoc filtered 20604 features

83 significant genes with 102 significant transcripts (OFDR: 0.05)

```
> dtu_res$FDR_table[order(dtu_res$FDR_table$gene), ]
```

	geneID	txID	gene	transcript
11033	Gnas	ENSMUST00000087871	9.677205e-64	1.000000e+00
11034	Gnas	ENSMUST00000109084	9.677205e-64	1.000000e+00
11035	Gnas	ENSMUST00000109085	9.677205e-64	4.366673e-16
11036	Gnas	ENSMUST00000109087	9.677205e-64	1.299696e-10
11037	Gnas	ENSMUST00000109096	9.677205e-64	1.000000e+00
11038	Gnas	ENSMUST00000133356	9.677205e-64	1.000000e+00
11039	Gnas	ENSMUST00000149016	9.677205e-64	1.000000e+00
11040	Gnas	ENSMUST00000156623	9.677205e-64	2.329646e-10
11041	Gnas	ENSMUST00000238267	9.677205e-64	1.000000e+00
11042	Gnas	ENSMUST00000238525	9.677205e-64	1.000000e+00
11043	Gnas	ENSMUST00000238660	9.677205e-64	1.000000e+00
22094	Rtn1	ENSMUST00000021497	6.254713e-29	0.000000e+00
22095	Rtn1	ENSMUST00000078505	6.254713e-29	0.000000e+00
16575	Ndufs2	ENSMUST00000013737	1.088462e-16	0.000000e+00
16576	Ndufs2	ENSMUST00000150108	1.088462e-16	0.000000e+00

With filtering and batch correction:

Drimseq retained 4048 of 10923 features and removed 6875.
Posthoc filtered 3416 features

293 significant genes with 414 significant transcripts (OFDR: 0.05)

```
> dtu_res$FDR_table[order(dtu_res$FDR_table$gene), ]
```

	geneID	txID	gene	transcript
1392	Gnas	ENSMUST00000087871	6.094618e-296	1.000000e+00
1393	Gnas	ENSMUST00000109084	6.094618e-296	1.000000e+00
1394	Gnas	ENSMUST00000109085	6.094618e-296	6.768349e-76
1395	Gnas	ENSMUST00000109087	6.094618e-296	9.165581e-50
1396	Gnas	ENSMUST00000109096	6.094618e-296	1.000000e+00
1397	Gnas	ENSMUST00000133356	6.094618e-296	1.000000e+00
1398	Gnas	ENSMUST00000149016	6.094618e-296	1.000000e+00
1399	Gnas	ENSMUST00000156623	6.094618e-296	1.000000e+00
1400	Gnas	ENSMUST00000238267	6.094618e-296	1.000000e+00
1401	Gnas	ENSMUST00000238525	6.094618e-296	1.000000e+00
1402	Gnas	ENSMUST00000238660	6.094618e-296	1.000000e+00
2205	Ndufs2	ENSMUST00000013737	5.163277e-38	0.000000e+00
2206	Ndufs2	ENSMUST00000150108	5.163277e-38	0.000000e+00
244	Ang	ENSMUST00000069011	4.964089e-27	0.000000e+00
245	Ang	ENSMUST00000171688	4.964089e-27	0.000000e+00
1434	Gpr137b	ENSMUST00000021738	1.420466e-14	0.000000e+00
1435	Gpr137b	ENSMUST00000220628	1.420466e-14	0.000000e+00

Top significant genes

```
> unique(dtu_res$FDR_table[order(dtu_res$FDR_table$gene), ]$geneID[1:260])
```

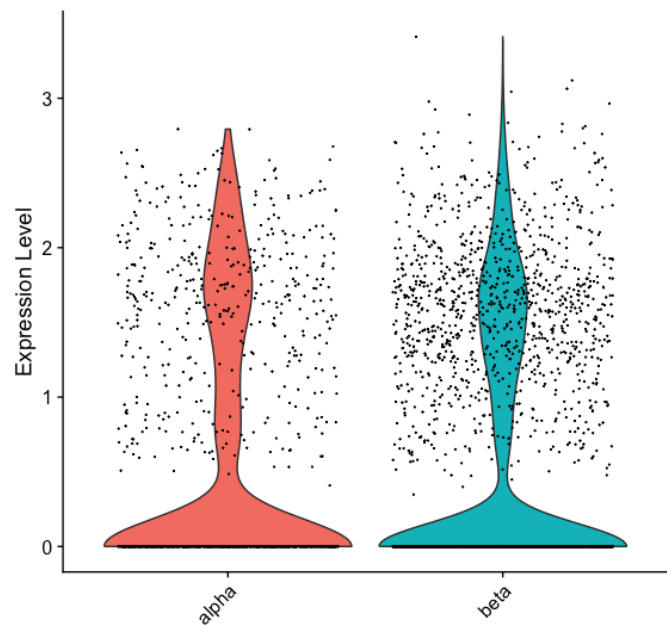
[1]	"Gnas"	"Ndufs2"	"Ang"	"Gpr137b"	"Hmgn3"	"Rsrp1"	"Fmo1"	"Ddc"	"Arpc5"
[10]	"Rnase4"	"Coro2b"	"Gas5"	"Pigk"	"Banp"	"Msmo1"	"Tmed9"	"Mettl26"	"Ddx50"
[19]	"Srpr"	"Sumo3"	"Myl6"	"Plgrkt"	"Atp2b1"	"Use1"	"Rpl22l1"	"Rpl13a"	"Med31"
[28]	"Paip2"	"Mat2a"	"Morf4l1"	"Anapc5"	"1500011B03Rik"	"Dusp1"	"Emc10"	"Wbp1"	"Gm42047"
[37]	"Eny2"	"Rpl10"	"Mast4"	"Csde1"	"Nisch"	"Cnot7"	"Auts2"	"H2-Ke6"	"Ccar1"
[46]	"Cep120"	"Chkb"	"Sdccag8"	"Abhd2"	"Snhg11"	"Rora"	"Fgf12"	"Hsp90aa1"	"Atp6v1b2"
[55]	"Stxbp1"	"Slc35b1"	"Sphkap"	"Rpl31"	"Rpl34"	"Kpna1"	"Lpl"	"Tsc22d1"	"Ank"
[64]	"Tmem60"	"Ypel3"	"Prdx2"	"Rpl28"	"Tmem219"	"Pip4p1"	"Snrnp70"	"Gnao1"	"Eif4a1"
[73]	"Dst"	"Tomm6"	"Ppp2r5a"	"Sec13"	"Yif1b"	"Sem1"	"Stk19"	"Rbis"	"Spty2d1"
[82]	"Snap25"	"Meg3"	"Atpif1"	"Deptor"	"Fkbp2"	"Pitpna"	"Rpl41"	"Cdc37l1"	"Abr"
[91]	"Bfar"	"Polr1d"	"Os9"	"Tram1"	"Hnf1aos1"	"Robo2"	"Cfap20"	"Avl9"	

Plotting

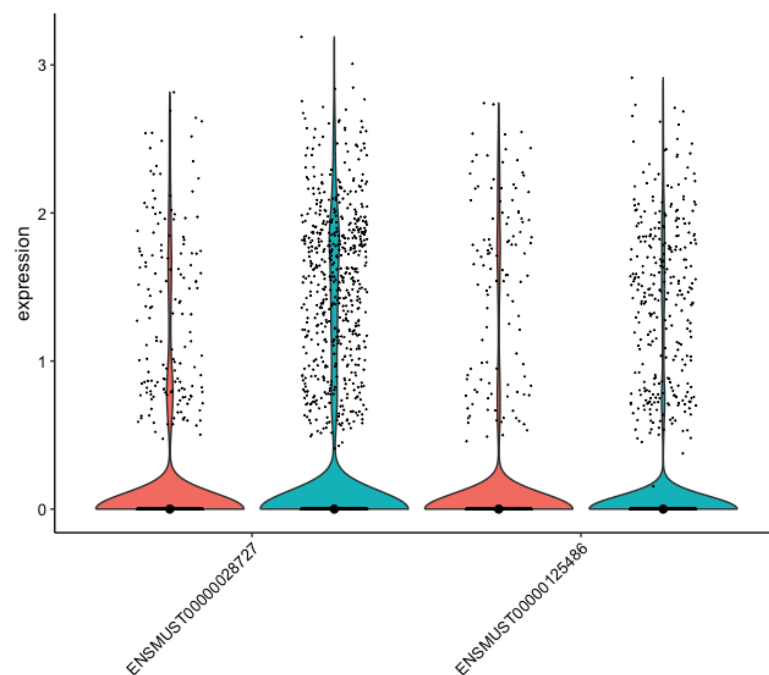
DGE

```
VlnPlot(object = so, features = "Snap25", group.by = "new_identity", cols = c("alpha" = "#F37066", "beta" = "#00B8C1"), assay = "RNA")
```

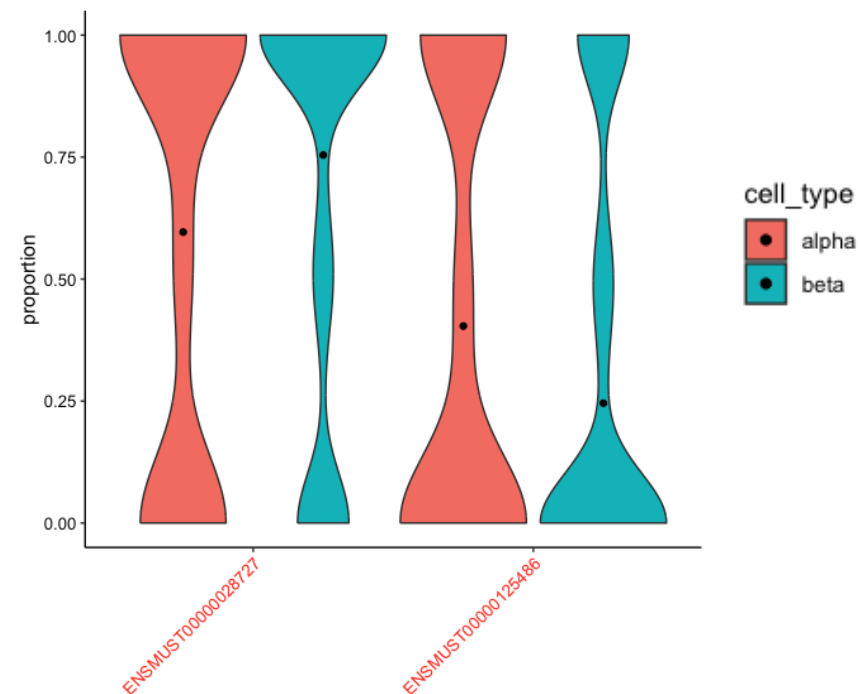
Snap25 gene expression



Snap25 transcript expression



Snap25 transcript usage

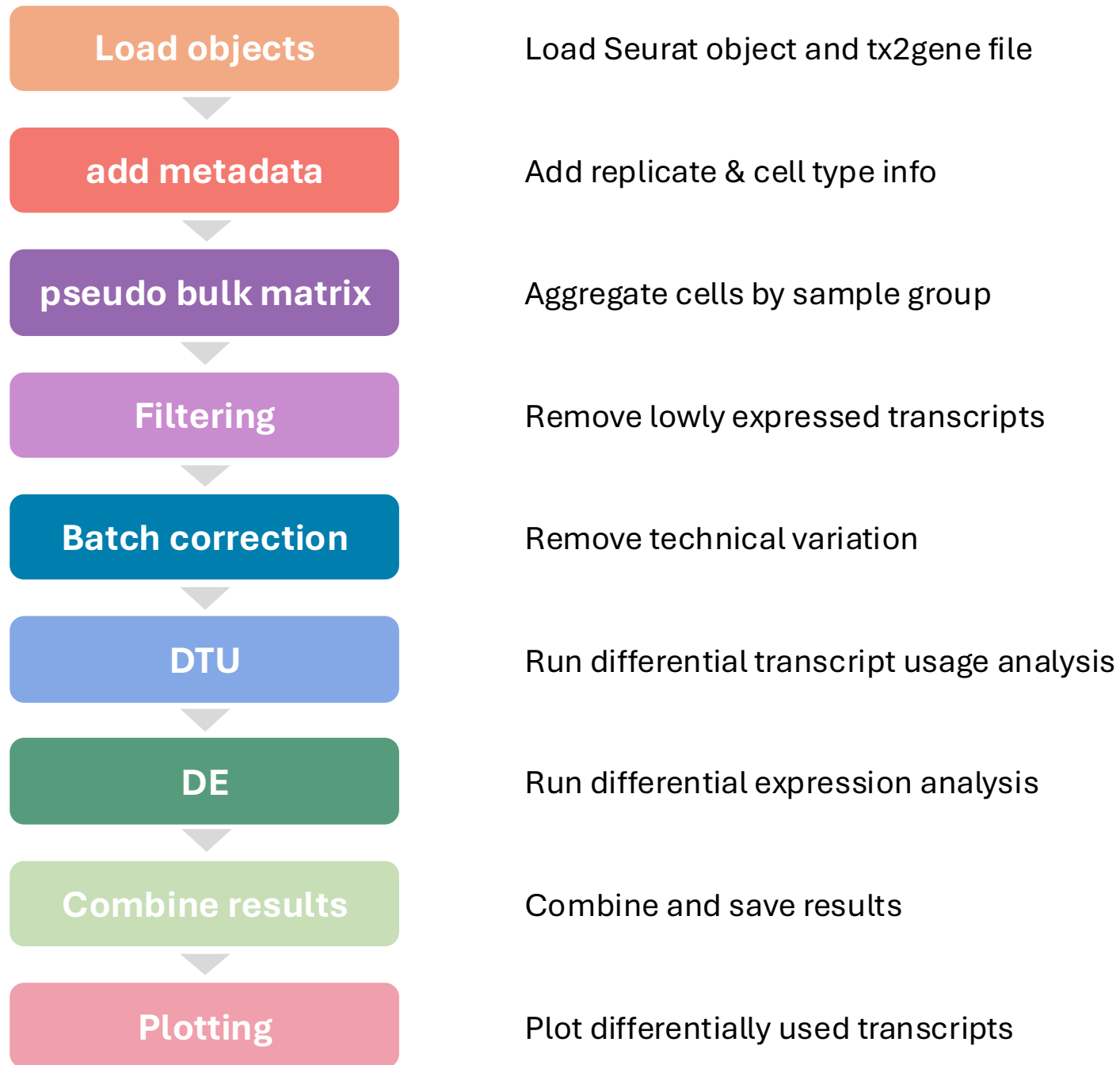


DTU

```
plot_dturtle_proportions_all(seurat_object = so, dturtle_object = dtu_res,  
                             gene_list = gene_to_plot, test_meta_col = "new_identity",  
                             test_ids = c("alpha", "beta"), color = c("alpha" = "#F37066", "beta" = "#00B8C1"))
```

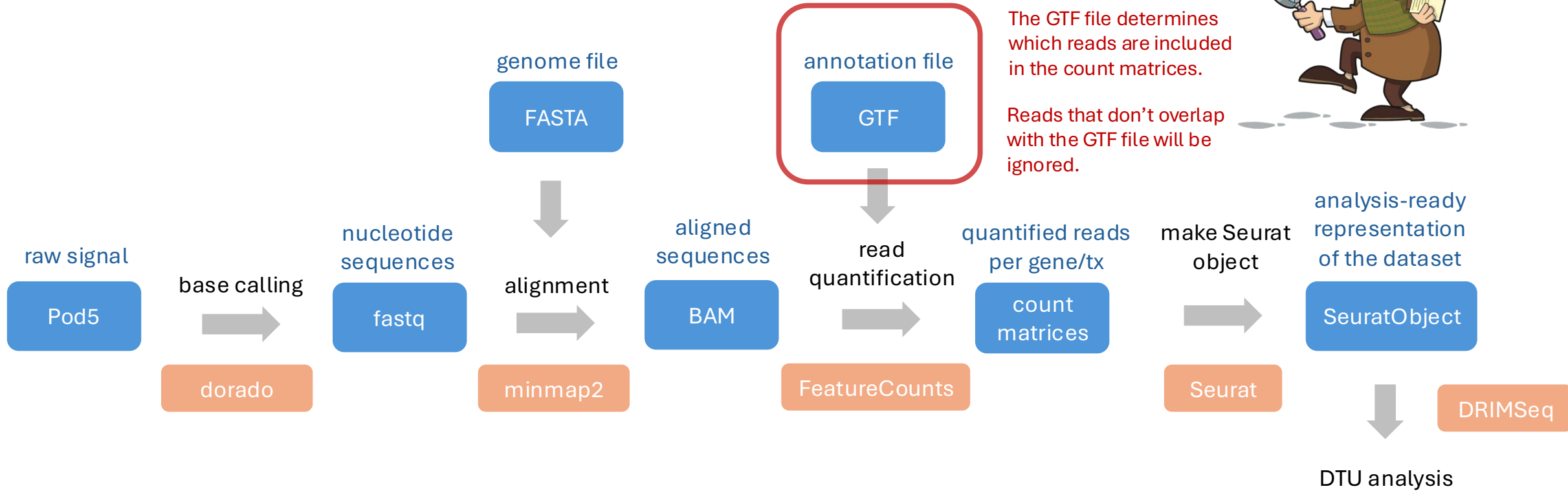
DTE

```
make_expression_violin(seurat_object = so,  
                       dturtle_object = dtu_res,  
                       gene_name = gene_to_plot, color = c("alpha" = "#F37066", "beta" = "#00B8C1"))
```



Analysis workflow

Tool File type



What is a GTF file?

A gene annotation file with:

- Genomics locations
- Transcript information
- Exon information

```
> head(gtf_file, n=5)
```

	seqnames	start	end	width	strand	source	type	score	phase	gene_id	gene_version	transcript_id	transcript_version	gene_type
1	chr1	3205901	3216344	10444	-	HAVANA	transcript	NA	NA	ENSMUSG00000051951	5	ENSMUST00000162897	1	protein_coding
2	chr1	3206523	3215632	9110	-	HAVANA	transcript	NA	NA	ENSMUSG00000051951	5	ENSMUST00000159265	1	protein_coding
3	chr1	3214482	3671498	457017	-	HAVANA	transcript	NA	NA	ENSMUSG00000051951	5	ENSMUST00000070533	4	protein_coding
4	chr1	3466587	3513553	46967	+	HAVANA	transcript	NA	NA	ENSMUSG00000089699	1	ENSMUST00000161581	1	lncRNA
5	chr1	3647309	3658904	11596	-	HAVANA	transcript	NA	NA	ENSMUSG00000102331	1	ENSMUST00000192692	1	lncRNA

	gene_name	transcript_type	transcript_name	level	transcript_support_level	mgid	havana_gene	havana_transcript	protein_id	tag
1	Xkr4	lncRNA	Xkr4-203	2		1 MGI:3528744	OTTMUSG00000026353.2	OTTMUST00000086625.1	<NA>	<NA>
2	Xkr4	lncRNA	Xkr4-202	2		1 MGI:3528744	OTTMUSG00000026353.2	OTTMUST00000086624.1	<NA>	<NA>
3	Xkr4	protein_coding	Xkr4-201	2		1 MGI:3528744	OTTMUSG00000026353.2	OTTMUST00000065166.1	ENSMUSP00000070648.4	CCDS
4	Gm1992	lncRNA	Gm1992-201	2		3 MGI:3780162	OTTMUSG00000026353.2	OTTMUST00000065165.1	<NA>	basic
5	Gm19938	lncRNA	Gm19938-201	2		1 MGI:5012123	OTTMUSG00000026353.2	OTTMUST00000065165.1	<NA>	basic

	ccdsid	ont
1	<NA>	<NA>
2	<NA>	<NA>
3	CCDS14803.1	<NA>
4	<NA>	<NA>
5	<NA>	<NA>

Recommended to make a simplified “tx2gene file from the gtf file”.

```
# Get all transcript names from gene:
tx2gene$transcript_id[tx2gene$gene_name == "G6pc2"]

# Get gene name from transcript:
tx2gene$gene_name[tx2gene$transcript_id == "ENSMUST00000049972"]
```

You can download GTF files from 10X Genomics’ website:

Mouse reference (GRCm39) - 2024-A

- Mouse reference (GRCm39) dataset required for Cell Ranger.

[Download Mouse Reference](#)

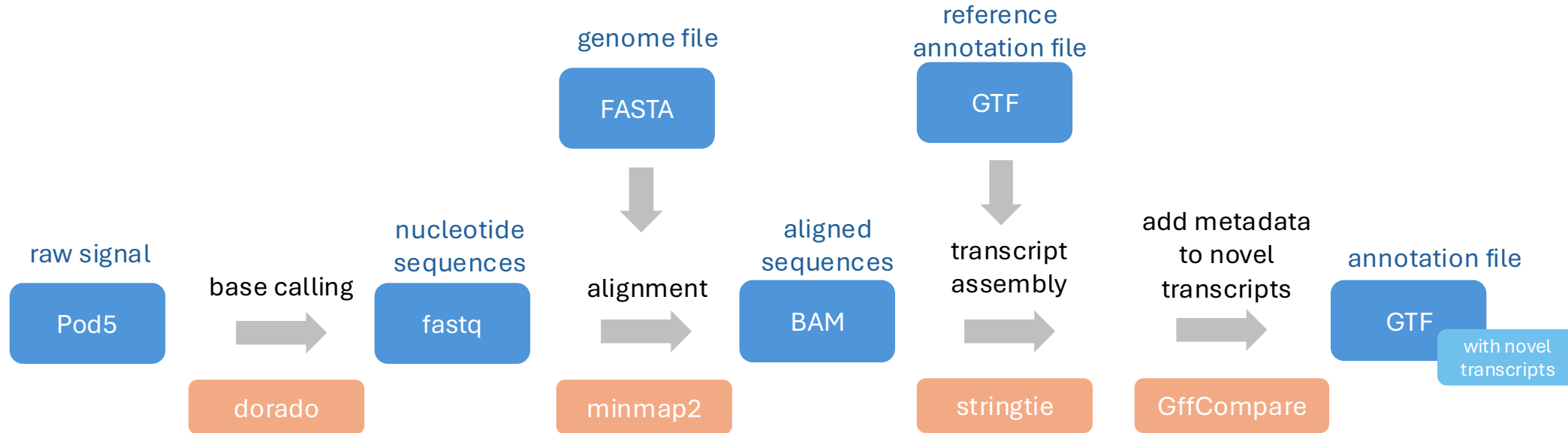
File size: 9.7 GB

md5sum: 37c51137ccaeabd4d151f80dc86ce0b3

curl wget

[Copy](#)

```
curl -O "https://cf.10xgenomics.com/supp/cell-exp/refdata-gex-GRCm39-2024-A"
```



=	complete, exact match of intron chain
c	contained in reference (intron compatible)
k	containment of reference (reverse containment)
m	retained intron(s), all introns matched or retained
n	retained intron(s), not all introns matched/covered
j	multi-exon with at least one junction match
e	single exon transfrag partially covering an intron, possible pre-mRNA fragment
o	other same strand overlap with reference exons
s	intron match on the opposite strand (likely a mapping error)
x	exonic overlap on the opposite strand (like o or e but on the opposite strand)
i	fully contained within a reference intron
y	contains a reference within its intron(s)
p	possible polymerase run-on (no actual overlap)
r	repeat (at least 50% bases soft-masked)
u	none of the above (unknown, intergenic)

StringTie generates a new transcript annotation, based on the reads. It may include entirely novel transcripts, new exons, or new combinations of exons. It will also merge GTF files from multiple samples.

GffCompare compares assembled transcripts to a reference annotation and reports, for each query transcript, reference overlaps found, along with their classification codes.